AI/ML DEEP DIVE

MLOps & Productionizing ML

From notebook to production — experiment tracking, data & model versioning, pipelines, CI/CD, serving patterns, containerisation, feature stores, drift monitoring, reproducibility, and LLMOps.

mlops · production · ml-lifecycle · model-serving · monitoring · ci-cd · llmops

Study Pack · Comprehensive reference & interview prep

CONTENTS

1. Why MLOps Matters: The Notebook-to-Production Gap
2. The ML Lifecycle
3. Experiment Tracking with MLflow
4. Data Versioning with DVC
5. Model Versioning and the Model Registry
6. ML Pipelines: Kubeflow, Prefect, and Airflow
7. CI/CD for Machine Learning
8. Model Serving Patterns
9. Containerisation with Docker for ML
10. Feature Stores
11. A/B Testing and Shadow Deployment
12. Monitoring and Drift Detection
13. Reproducibility in Machine Learning
14. LLMOps: Production LLM Systems
15. Q&A: Test Your Understanding
16. MLOps Tooling Landscape

Why MLOps Matters: The Notebook-to-Production Gap

MLOps is the discipline of taking machine learning models from experimental notebooks to reliable, monitored, and continuously-improving production systems. The gap between these two worlds is where most ML projects fail.

A common misconception is that the hard part of building an ML product is training the model. In practice, the model itself is a small fraction of the system. Google's landmark 2015 paper "*Hidden Technical Debt in Machine Learning Systems*" illustrates this: the ML code sits in a tiny box at the centre, surrounded by data collection, feature engineering, serving infrastructure, monitoring, process management, and configuration. All of that surrounding machinery must be built, versioned, and operated. MLOps is the set of practices, tooling, and culture that makes this sustainable.

Core tension

Traditional software is *code + config*. ML systems are *code + config + data + model weights*. The last two mutate independently over time — data distributions shift, labels arrive late, retraining changes model behaviour — requiring versioning and monitoring that classical DevOps was never designed for.

THE 3 AXES OF FAILURE

Data: training-serving skew, schema drift, missing features in prod.

Model: silent performance degradation, concept drift, stale embeddings.

Operations: no rollback plan, no canary, inconsistent environments.

WHAT MLOPS ADDS TO DEVOPS

Experiment tracking & reproducibility, data & model versioning, feature stores, drift monitoring, retraining pipelines, model governance, and LLM-specific concerns (prompt versioning, cost tracking).

MATURITY LEVELS

Level 0: manual notebooks, ad-hoc deployment.

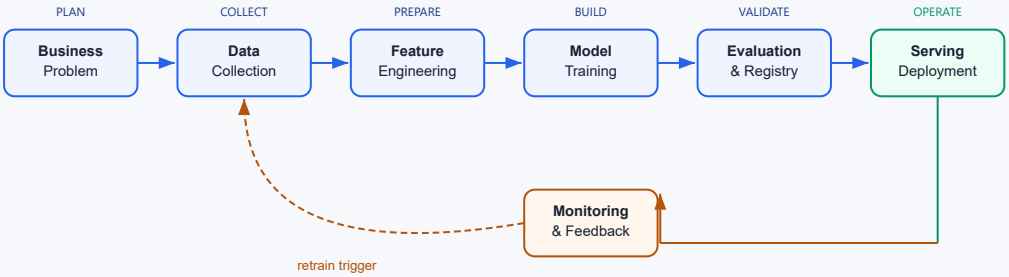
Level 1: automated pipelines, experiment tracking.

Level 2: full CI/CD for models, automated retraining triggers.

> ⚠ **Stat**
>
> Industry surveys consistently show that over 85 % of ML projects never reach production, and of those that do, the median time from first prototype to live serving is 6–18 months. MLOps practices compress that timeline by an order of magnitude.

The ML Lifecycle

Unlike classical software, the ML lifecycle is a *loop*, not a pipeline. Every production system eventually triggers a feedback signal that demands retraining, which starts the cycle again.



ML Lifecycle — the loop from business problem through data, features, training, evaluation and serving, with monitoring triggering continuous retraining.

Key insight: phases after deployment (monitoring, feedback collection, retraining triggers) receive less engineering attention than pre-deployment phases, yet they determine whether a system stays accurate over its entire lifespan.

Experiment Tracking with MLflow

Every model training run generates dozens of parameters, metrics, and artefacts. Without systematic tracking, reproducing last quarter's best result is nearly impossible. MLflow is the de-facto open-source standard for experiment tracking.

MLflow organises work around four core concepts: **Experiments** (a named project), **Runs** (a single training execution with its own ID), **Params** (hyperparameters logged at the start), **Metrics** (time-series scalars like loss or F1), and **Artefacts** (files: model binaries, confusion matrices, vocabulary files). All runs within an experiment are queryable and comparable in the MLflow UI.

PYTHON

```python
import mlflow
import mlflow.sklearn
from sklearn.ensemble import RandomForestClassifier

mlflow.set_experiment("fraud-detection-v2")

with mlflow.start_run(run_name="rf-baseline"):
    params = {"n_estimators": 200, "max_depth": 8, "random_state": 42}
    mlflow.log_params(params)

    clf = RandomForestClassifier(**params)
    clf.fit(X_train, y_train)

    precision, recall, f1, _ = precision_recall_fscore_support(
        y_test, clf.predict(X_test), average="binary"
    )
    mlflow.log_metrics({"precision": precision, "recall": recall, "f1": f1})

    # Log the model with its signature for schema enforcement
    signature = mlflow.models.infer_signature(X_train, clf.predict(X_train))
    mlflow.sklearn.log_model(clf, "model", signature=signature)

    mlflow.log_artifact("feature_importance.png")
    print(f"Run ID: {mlflow.active_run().info.run_id}")
```

Best Practice

Always log a **model signature** (input/output schema) with every run. MLflow will enforce this schema at serving time, catching training-serving skew early rather than silently returning wrong predictions.

MLflow Model Registry

After tracking runs, promising models are promoted into the **Model Registry** — a versioned catalogue with a lifecycle: *None → Staging → Production → Archived*. The registry is the single source of truth for which model version is live.

PYTHON

```python
from mlflow.tracking import MlflowClient

client = MlflowClient()

# Register a run's model artifact under a named model
result = mlflow.register_model(
    model_uri=f"runs:/{run_id}/model",
    name="fraud-detector"
)

# Transition to Staging after validation
client.transition_model_version_stage(
    name="fraud-detector",
    version=result.version,
    stage="Staging",
    archive_existing_versions=False
)

# Promote to Production — this is typically automated via CI/CD
client.transition_model_version_stage(
    name="fraud-detector",
    version=result.version,
    stage="Production",
    archive_existing_versions=True   # auto-archive old prod version
)
```

MLflow 2.x

MLflow 2.x replaces stage aliases with explicit `@champion` / `@challenger` tags, making multi-deployment strategies cleaner. The concepts remain the same; only the API surface changes.

Data Versioning with DVC

Git tracks code. DVC (Data Version Control) tracks data, models, and the pipelines that connect them. Together they give complete, reproducible lineage for every experiment.

DVC stores large files in a remote storage backend (S3, GCS, Azure Blob, SSH) while keeping a tiny `.dvc` pointer file in Git. `git checkout` a branch and `dvc pull` to materialise the exact dataset that branch was trained on — even six months later.

BASH

```bash
# Initialize DVC alongside Git
dvc init
git add .dvc
git commit -m "chore: init DVC"

# Track a large dataset
dvc add data/raw/transactions.parquet
git add data/raw/transactions.parquet.dvc .gitignore
git commit -m "data: add raw transactions v1"

# Configure remote storage (AWS S3)
dvc remote add -d myremote s3://my-ml-bucket/dvc-store
dvc push          # upload to S3

# Later: reproduce the exact dataset on another machine
git checkout v1.2.0
dvc pull          # fetches the exact data version
```

DVC Pipelines (dvc.yaml)

DVC can define a **directed acyclic graph (DAG)** of data transformation stages. Running `dvc repro` executes only the stages whose inputs have changed — like Make for data science.

YAML

```yaml
stages:
  prepare:
    cmd: python src/prepare.py
    deps:
      - data/raw/transactions.parquet
      - src/prepare.py
    outs:
      - data/processed/features.parquet

  train:
    cmd: python src/train.py --config params.yaml
    deps:
      - data/processed/features.parquet
      - src/train.py
    params:
      - params.yaml:
          - train.learning_rate
          - train.n_estimators
    outs:
      - models/model.pkl
    metrics:
      - metrics/eval.json:
          cache: false
```

✦ **DVC + MLflow**

Use DVC for *data and pipeline* versioning and MLflow for *experiment* tracking. They complement each other: DVC stages can call MLflow-instrumented training scripts, giving you full lineage from raw data to registered model.

Model Versioning and the Model Registry

A model registry is the central catalogue that maps model names to versioned, validated artefacts, bridging the gap between data scientists and deployment engineers.

Beyond MLflow's built-in registry, production teams use dedicated platforms. The registry must answer three questions: *which version is in production right now?*, *who approved it and when?*, and *where is the artefact stored?*

WHAT A REGISTRY STORES

- Model artefact URI (S3 path, container image digest)
- Framework and serialisation format (pickle, ONNX, TorchScript, SavedModel)
- Input/output schema (feature names, dtypes, expected shape)
- Training run lineage (MLflow run ID, Git SHA, dataset version)
- Evaluation metrics, approval audit trail

REGISTRY LIFECYCLE STAGES

Candidate: newly logged from a training run, not yet validated.

Staging: passed automated tests, awaiting human sign-off or A/B evaluation.

Production: serving live traffic; only one version per alias at a time.

Archived: replaced but retained for rollback. Auto-archived by policy after N days.

Model Serialisation Formats

Format	Portability	Performance	Best For
Pickle / Joblib	Python-only	Baseline	sklearn, quick iteration
ONNX	Cross-language, cross-runtime	Up to 3× faster inference	Cross-platform deployment, edge
TorchScript	C++ / mobile	Good; graph-optimised	PyTorch to production or mobile
TF SavedModel	TensorFlow Serving, TF.js	Excellent with TF Serving	TensorFlow ecosystem
MLflow pyfunc	Any environment with mlflow	Wrapper overhead	Framework-agnostic serving API
Container image	Any Kubernetes cluster	Depends on inner runtime	Custom preprocessing, complex deps

ML Pipelines: Kubeflow, Prefect, and Airflow

Manual, notebook-driven training cannot scale. ML pipelines automate every step from data ingestion through model deployment, making retraining a push-button (or fully automatic) operation.

> **Key distinction**
> Data engineering pipelines (Airflow, Spark) orchestrate data movement. ML pipelines orchestrate *training runs* and need ML-specific primitives: GPU scheduling, experiment tracking integration, distributed training, and artefact passing between steps.

Kubeflow Pipelines

Kubeflow Pipelines (KFP) runs on Kubernetes and compiles Python-defined DAGs into Argo Workflow YAML. Each step executes in its own container, enabling per-step resource requests (CPU, GPU, memory).

PYTHON

```python
from kfp import dsl
from kfp.components import create_component_from_func

def preprocess(data_path: str, output_path: str):
    import pandas as pd
    df = pd.read_parquet(data_path)
    df.dropna().to_parquet(output_path)

def train(features_path: str, model_output: str, n_estimators: int = 200):
    import mlflow, joblib
    from sklearn.ensemble import RandomForestClassifier
    # ... training code
    joblib.dump(clf, model_output)

preprocess_op = create_component_from_func(preprocess, base_image="python:3.11-slim")
train_op      = create_component_from_func(train,      base_image="ml-training:latest")

@dsl.pipeline(name="fraud-pipeline", description="Weekly retraining")
def fraud_pipeline(data_path: str = "s3://bucket/raw/"):
    prep = preprocess_op(data_path=data_path, output_path="/tmp/features.parquet")
    train_op(features_path=prep.outputs["output_path"], model_output="/tmp/model.pkl")
```

Apache Airflow for ML

Airflow excels at scheduling and dependency management. Its `KubernetesPodOperator` and `DockerOperator` allow ML tasks to run in isolated containers. Airflow is the right choice when you already have data engineering pipelines in Airflow and want to attach retraining triggers to data availability events.

Prefect

Prefect 2.x uses native Python functions decorated with `@flow` and `@task`. It runs locally, on Prefect Cloud, or on Kubernetes. Its dynamic scheduling, automatic retries, and result caching make it popular for teams that find Airflow's XML/YAML overhead excessive.

PYTHON

```python
from prefect import flow, task
from prefect.tasks import task_input_hash
from datetime import timedelta

@task(cache_key_fn=task_input_hash, cache_expiration=timedelta(hours=24))
def load_features(dataset_version: str):
    # load and return features — result is cached by version hash
    ...

@task(retries=3, retry_delay_seconds=30)
def train_model(features, params: dict):
    ...

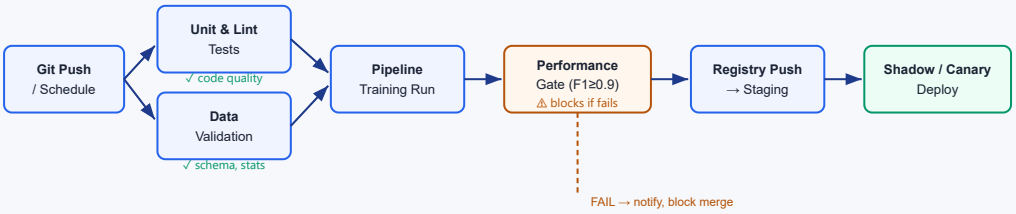
@flow(name="weekly-retrain", log_prints=True)
def retrain_pipeline(dataset_version: str = "latest"):
    features = load_features(dataset_version)
    model    = train_model(features, params={"n_estimators": 300})
    evaluate_and_register(model)

if __name__ == "__main__":
    retrain_pipeline(dataset_version="2026-09-28")
```

Orchestrator	Strength	Weakness	ML-native?
Kubeflow Pipelines	Container-native, GPU scheduling, KFP SDK	Kubernetes required, complex setup	Yes — built for ML
Apache Airflow	Mature, huge operator library, broad community	Verbose DAG syntax, no native ML artefacts	Partial (via operators)
Prefect 2.x	Pure Python, easy local dev, caching, Prefect Cloud	Smaller operator ecosystem than Airflow	Partial (via tasks)
Metaflow (Netflix)	Notebook-to-cloud, versioning built-in, S3 integration	AWS-centric historically	Yes — built for DS teams
ZenML	Stack-agnostic abstractions, built-in lineage, registry	Newer ecosystem	Yes — MLOps-first design

CI/CD for Machine Learning

Classical CI/CD tests code correctness. ML CI/CD also tests *data quality*, *model performance gates*, and *serving infrastructure* — only promoting a new model version when all three pass.



ML CI/CD Pipeline — code and data validations run in parallel; only when both pass does training execute, followed by a performance gate before registry promotion.

GitHub Actions for ML CI

YAML

```yaml
name: ML CI Pipeline
on:
  push:
    branches: [main]
  schedule:
    - cron: "0 3 * * 1"   # weekly Monday 03:00 UTC

jobs:
  data-validation:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.11" }
      - run: pip install great-expectations dvc[s3]
      - run: dvc pull data/processed/features.parquet
      - run: python src/validate_data.py   # raises if schema/stats fail

  train-and-gate:
    needs: data-validation
    runs-on: [self-hosted, gpu]
    steps:
      - run: python src/train.py
      - run: python src/evaluate.py --threshold 0.90   # exits non-zero if below
      - name: Register to Staging
        run: python src/register_model.py --stage Staging
        env:
          MLFLOW_TRACKING_URI: ${{ secrets.MLFLOW_URI }}
```

✦ **ML-specific tests to add**
(1) *Data schema tests*: assert column types, value ranges, null rates using Great Expectations or Pandera. (2) *Model invariance tests*: small perturbations to input should not cause wild prediction swings. (3) *Baseline comparison*: new model must beat current prod model's metric by at least Δ threshold. (4) *Load test*: serving container must meet latency SLA (P99 < 200 ms) under expected QPS.

Model Serving Patterns

How a model is served determines its latency, throughput, cost, and operational complexity. The right pattern depends on the use case — not every application needs a real-time REST endpoint.

BATCH INFERENCE

Pattern: Scheduled job reads a dataset, runs predictions, writes results to a database or file.

Latency: Minutes to hours.

Cost: Very low — spot instances, no always-on server.

Best for: Nightly recommendations, fraud scoring on closed transactions, churn scoring.

Tools: Spark, Ray, AWS Batch, Databricks Jobs.

ONLINE / REAL-TIME REST

Pattern: Always-on HTTP server; clients send a feature payload and receive a prediction synchronously.

Latency: Single-digit to low-double-digit ms.

Cost: Continuous compute; auto-scaling critical.

Best for: Real-time fraud, search ranking, ad click prediction, chatbots.

Tools: BentoML, TorchServe, TF Serving, Triton Inference Server, FastAPI.

STREAMING INFERENCE

Pattern: Model consumes events from a message queue (Kafka/Kinesis) and emits prediction events downstream.

Latency: Near-real-time (seconds).

Cost: Moderate; event-driven scale.

Best for: IoT anomaly detection, session-level personalisation, continuous log scoring.

Tools: Flink ML, Kafka Streams + model sidecar, Ray Streaming.

Building a REST Serving Endpoint with BentoML

PYTHON

```python
import bentoml
from bentoml.io import NumpyNdarray, JSON
import numpy as np

# Save a trained model into the BentoML model store
bentoml.sklearn.save_model("fraud_detector", clf,
    signatures={"predict": {"batchable": True, "batch_dim": 0}},
    metadata={"training_run_id": run_id, "f1": 0.947}
)

# Define a Service
svc = bentoml.Service("fraud_svc", runners=[
    bentoml.sklearn.get("fraud_detector:latest").to_runner()
])

@svc.api(input=JSON(), output=JSON())
async def predict(features: dict) -> dict:
    arr = np.array([[features[k] for k in FEATURE_NAMES]])
    prob = await fraud_runner.predict.async_run(arr)
    return {"fraud_probability": float(prob[0, 1]),
            "decision": "BLOCK" if prob[0, 1] > 0.85 else "ALLOW"}
```

Triton Inference Server

NVIDIA Triton is the production standard for GPU-accelerated serving. It supports ONNX, TensorRT, TorchScript, TF SavedModel, and Python backends in a single server, with dynamic batching, concurrent model instances, and model ensembling. Triton's dynamic batcher accumulates incoming requests into mini-batches transparently, dramatically improving GPU utilisation without client-side changes.

> **Latency vs Throughput**
> Online endpoints must minimise P99 latency. Batch endpoints maximise throughput. Streaming endpoints optimise for end-to-end event lag. These goals conflict — design each serving tier with only one objective in mind and measure the right SLI.

Containerisation with Docker for ML

Docker provides the reproducible environment that bridges a data scientist's laptop and a Kubernetes cluster. ML containers have unique requirements: large base images, GPU drivers, CUDA versions, and efficient layer caching.

BASH

```
# Multi-stage ML Dockerfile
# Stage 1: build/install deps (not shipped to prod)
FROM python:3.11-slim AS builder
WORKDIR /build
COPY requirements.txt .
RUN pip install --prefix=/install --no-cache-dir -r requirements.txt

# Stage 2: lean runtime image
FROM python:3.11-slim AS runtime
WORKDIR /app
COPY --from=builder /install /usr/local
COPY src/ ./src/
COPY models/ ./models/

# Non-root user for security
RUN useradd -m mluser
USER mluser

ENV MODEL_PATH=/app/models/fraud_detector.pkl
EXPOSE 8080
HEALTHCHECK --interval=30s CMD curl -f http://localhost:8080/health || exit 1
CMD ["uvicorn", "src.serve:app", "--host", "0.0.0.0", "--port", "8080"]
```

GPU-Enabled Containers

BASH

```
# CUDA base image — pin the exact CUDA + cuDNN version
FROM nvcr.io/nvidia/pytorch:24.05-py3
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
# NVIDIA runtime must be configured in daemon.json on the host:
# {"runtimes": {"nvidia": {"path": "nvidia-container-runtime"}}}
# docker run --gpus all --rm -it my-ml-image:latest
```

✦ **Layer caching strategy**

Order Dockerfile instructions from least-frequently-changed to most: (1) OS packages, (2) heavy Python deps (torch, tensorflow), (3) project dependencies (requirements.txt), (4) application code. This maximises cache hits and keeps rebuild times under a minute for code-only changes.

⚠ **Model size anti-pattern**

Never `COPY` large model weights into the Docker image. Images become multi-GB, CI/CD slows to a crawl, and you cannot update the model without rebuilding. Instead, mount weights from a volume or pull from a model registry at container startup using the model path as an environment variable.

Feature Stores

A feature store is a centralised repository for computed features. It solves the two biggest sources of training-serving skew: feature reuse across teams, and identical feature computation in both the training pipeline and the real-time serving path.

Without a feature store, team A writes an "average spend last 30 days" feature in Spark for training, and team B writes a slightly different SQL version for the API. The model is trained on one definition and scored on another. This silent mismatch causes unexplained production performance degradation.

OFFLINE STORE

Stores historical feature values at scale — typically a columnar format like Parquet on S3 or a data warehouse (BigQuery, Redshift). Used for model training and batch inference. Supports point-in-time correct joins to prevent label leakage.

Key requirement: point-in-time retrieval — "what were the feature values for entity X at time T?" — essential for backtesting and avoiding leakage.

ONLINE STORE

Low-latency (sub-millisecond) key-value store — typically Redis, DynamoDB, or Bigtable. Stores the *latest* feature values for real-time serving. Features are materialised from the offline store via a batch/streaming job.

Key requirement: P99 lookup latency under 5 ms to not dominate the model's own inference time.

PYTHON

```python
from feast import FeatureStore, Entity, FeatureView, Field
from feast.types import Float64, Int64
import pandas as pd

store = FeatureStore(repo_path=".")

# Training: retrieve historical features with point-in-time correctness
training_df = store.get_historical_features(
    entity_df=pd.DataFrame({
        "customer_id": [101, 202, 303],
        "event_timestamp": pd.to_datetime(["2026-09-01", "2026-09-05", "2026-09-10"])
    }),
    features=["customer_stats:avg_spend_30d", "customer_stats:txn_count_7d"]
).to_df()

# Serving: retrieve latest features from online store (Redis)
online_features = store.get_online_features(
    features=["customer_stats:avg_spend_30d", "customer_stats:txn_count_7d"],
    entity_rows=[{"customer_id": 101}]
).to_dict()
```

Popular Feature Stores 2026

Feast (open-source, cloud-agnostic), **Tecton** (managed, streaming + batch), **Hopsworks** (open-source with enterprise tier), **Vertex AI Feature Store** (GCP-native), **SageMaker Feature Store** (AWS-native), **Databricks Feature Engineering** (Delta Lake-backed).

A/B Testing and Shadow Deployment

Deploying a new model version directly to 100 % of production traffic is high-risk. Progressive deployment patterns — shadow, canary, and A/B — reduce the blast radius of a bad model.



Shadow Mode

The new model receives a copy of every production request, runs inference, and logs results — but its predictions are **not returned to users**. Engineers compare shadow predictions vs production predictions offline to detect regressions, latency differences, or error rates before any user impact.

Canary Deployment

Route a small percentage (1–10 %) of live traffic to the new model version. Monitor business metrics (conversion rate, error rate, revenue per session) on the canary slice versus the baseline. Automated rollback triggers fire if the canary metric degrades beyond a threshold.

A/B Testing

Split traffic into controlled groups (A = current model, B = new model). Randomise assignment at the user or session level to avoid confounders. Run for sufficient time (typically until statistical power is achieved — a minimum detectable effect at 80 % power). Analyse with a two-proportion z-test or a Bayesian framework.

PYTHON

```python
from scipy import stats
import numpy as np

# Observed business metric: conversion rate
conversions_a = 1240; visitors_a = 12400   # 10.0 %
conversions_b = 1380; visitors_b = 12300   # 11.2 %

p_a = conversions_a / visitors_a
p_b = conversions_b / visitors_b
p_pool = (conversions_a + conversions_b) / (visitors_a + visitors_b)
se = np.sqrt(p_pool * (1 - p_pool) * (1/visitors_a + 1/visitors_b))
z = (p_b - p_a) / se
p_value = stats.norm.sf(abs(z)) * 2   # two-tailed

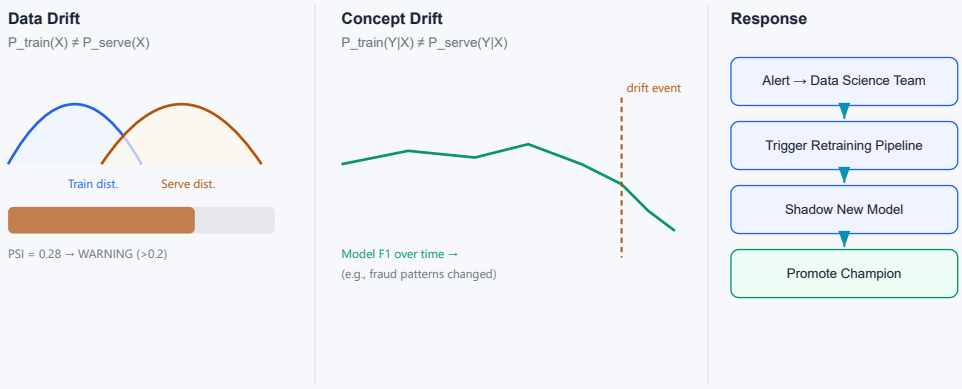
print(f"Lift: {(p_b - p_a) / p_a * 100:.1f} %")
print(f"p-value: {p_value:.4f}")
print("Significant!" if p_value < 0.05 else "Not significant")
```

⚠ **A/B pitfalls in ML**

(1) *Sample ratio mismatch*: traffic split is not what you configured — indicates a routing bug. (2) *Novelty effect*: users respond to anything new; wait for behaviour to stabilise. (3) *Cannibalism*: A/B groups interact (social features, inventory). (4) *Metric alignment*: optimise the *business* metric, not the proxy ML metric used during training.

Monitoring and Drift Detection

A model's accuracy silently decays after deployment as the world changes. Drift detection is the automated system that notices this decay before business metrics suffer.



Drift Monitoring — data drift (distribution shift in inputs) and concept drift (degrading model accuracy) each trigger an automated response chain ending in model replacement.

Types of Drift

Type	Definition	Detection Method	Example
Data / Covariate drift	Input feature distribution shifts: P(X) changes	PSI, KL-divergence, KS test, chi-squared	New demographics entering market, seasonal patterns
Concept drift	Relationship between features and target changes: P(Y|X) changes	Monitor accuracy on labelled samples, performance metrics	Fraud tactics evolve, consumer preferences shift
Label drift	Target variable distribution changes: P(Y) changes	Track prediction distribution, compare to historical label rates	Class imbalance worsens after a business event
Schema drift	Feature names, types, or cardinalities change upstream	Schema validation on every batch, Great Expectations	Upstream team renames a column
Prediction drift	Model output distribution shifts even without ground truth	Monitor output histogram, Jensen-Shannon divergence	Serving skew or upstream feature pipeline change

Drift Detection with Evidently AI

PYTHON

```python
import pandas as pd
from evidently.report import Report
from evidently.metric_preset import DataDriftPreset, ModelPerformancePreset

reference_df = pd.read_parquet("data/reference_month.parquet")
current_df   = pd.read_parquet("data/current_week.parquet")

# Data drift report — PSI and KS test for every feature
report = Report(metrics=[DataDriftPreset(stattest="psi", threshold=0.2)])
report.run(reference_data=reference_df, current_data=current_df)

# Access results programmatically for alerting
result = report.as_dict()
drifted_features = [
    f["column_name"]
    for f in result["metrics"][0]["result"]["drift_by_columns"].values()
    if f["drift_detected"]
]

if drifted_features:
    send_alert(f"Data drift detected in: {drifted_features}")
    trigger_retraining_pipeline()

report.save_html("reports/drift_report.html")
```

✓ **Golden signals for ML monitoring**
Borrow from SRE: (1) *Latency* — P50/P99 inference time. (2) *Traffic* — requests/sec, prediction volume. (3) *Errors* — prediction failures, null inputs, schema violations. (4) *Saturation* — GPU/CPU utilisation, queue depth. Add ML-specific: (5) *Prediction distribution* — output histogram shift. (6) *Feature drift* — PSI per feature. (7) *Business metrics* — downstream KPIs tied to model outputs.

Reproducibility in Machine Learning

Reproducibility means any team member can re-run any experiment from any point in time and obtain the same result. Without it, debugging production issues or auditing model decisions is nearly impossible.

Reproducibility in ML is harder than in software because it requires versioning four independent artefacts simultaneously: **code** (Git SHA), **data** (DVC commit), **environment** (Docker image digest or conda lock file), and **hyperparameters** (MLflow params). Losing any one of these makes exact reproduction impossible.

PYTHON

```python
import random, numpy as np, torch, os

def set_global_seed(seed: int = 42):
    """Fix all random sources for reproducible training."""
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    # Make CuDNN deterministic (trades some speed for reproducibility)
    torch.backends.cudnn.deterministic = True
    torch.backends.cudnn.benchmark = False
    os.environ["PYTHONHASHSEED"] = str(seed)

set_global_seed(42)

# Log the seed in MLflow so it's always captured
mlflow.log_param("random_seed", 42)
```

ENVIRONMENT PINNING

Use `pip-compile` (pip-tools) or `conda lock` to generate a fully-resolved, hash-pinned dependency file. `requirements.in` expresses intent; `requirements.txt` pins the exact resolved tree. Commit both files.

BASH

```bash
pip-compile requirements.in --generate-hashes -o requirements.txt
# Locks every transitive dep with SHA-256 hashes
```

DATA LINEAGE

Every model artefact in the registry should trace back to: the DVC data commit hash, the Git code SHA, the MLflow run ID, and the Docker image digest used f training. Many teams embed this in model metadata at log-time so the registry shows complete lineage.

PYTHON

```python
mlflow.log_params({
    "git_sha":     subprocess.check_output(
                     ["git", "rev-parse", "HEAD"]).decode().strip()
    "dvc_commit": subprocess.check_output(
                     ["dvc", "params", "diff"]).decode().strip(),
    "docker_image": os.getenv("IMAGE_DIGEST"),
})
```

⚠ **Non-determinism sources to eliminate**

(1) Unset random seeds. (2) Non-deterministic GPU ops (cuBLAS, cuDNN default). (3) Multi-threaded data loading without fixed worker seeds. (4) Floating-point non-associativity in distributed training. (5) Time-dependent features computed at train time (use a fixed reference timestamp). (6) External data sources queried live (snapshot data before training).

LLMOps: Production LLM Systems

Large language models in production introduce an entirely new set of operational concerns beyond classical MLOps: prompt versioning, non-deterministic outputs, hallucination risk, cost at scale, RAG pipeline monitoring, and evaluation frameworks that cannot rely on a single scalar metric.

Prompt Versioning

A prompt is code. Changing a system prompt without versioning is the LLM equivalent of deploying untested changes to production. Every prompt change must be: versioned in Git, tested against a regression evaluation suite, and rolled back independently of the model version.

PYTHON

```python
from langfuse import Langfuse

langfuse = Langfuse()

# Pull a versioned prompt from Langfuse Prompt Management
prompt_template = langfuse.get_prompt(
    name="contract-summariser",
    version=12,          # pin the exact version in prod
    label="production"   # or use labels for stage-based routing
)

# Compile with variables
final_prompt = prompt_template.compile(
    document=document_text,
    max_length="200 words",
    language="English"
)

# All calls are automatically traced to the prompt version
generation = langfuse.generation(
    name="summarise",
    model="claude-sonnet-4-6",
    input=final_prompt,
    prompt=prompt_template   # links trace to prompt version
)
```

RAG Pipeline Monitoring

A RAG system has multiple failure modes invisible to traditional metrics: poor retrieval recall (right answer not in retrieved context), context window overflow, stale vector embeddings after document updates, and answer hallucinations that contradict the retrieved passages.

RAG-SPECIFIC METRICS

- **Retrieval Recall@K** — what fraction of relevant chunks were returned in the top-K results?
- **Context Relevance** — are retrieved chunks actually useful for the question?
- **Faithfulness / Groundedness** — does the answer contradict the retrieved context?
- **Answer Relevance** — does the answer address the original question?
- **Embedding drift** — are document embeddings stale relative to the corpus?

RAG MONITORING STACK

Tracing: LangSmith, Langfuse, or Phoenix — capture every retrieval call, chunk, and generation with full input/output logging.

Evaluation: RAGAS framework computes retrieval and generation metrics automatically using an LLM-as-judge approach.

Embedding drift: Compare cosine similarity distributions in your vector index weekly against a reference period.

Cost: Track tokens per query (prompt + completion) to forecast spend; set per-user budget guardrails.

PYTHON

```python
from ragas import evaluate
from ragas.metrics import (
    faithfulness, answer_relevancy,
    context_precision, context_recall
)
from datasets import Dataset

# Build evaluation dataset (questions, contexts, answers, ground truths)
eval_data = Dataset.from_dict({
    "question":    questions,
    "contexts":    retrieved_contexts,   # list of lists
    "answer":      generated_answers,
    "ground_truth": reference_answers
})

result = evaluate(
    dataset=eval_data,
    metrics=[faithfulness, answer_relevancy, context_precision, context_recall]
)

print(result)
# faithfulness: 0.87 | answer_relevancy: 0.91
# context_precision: 0.74 | context_recall: 0.82
```

LLM Evaluation in Production

LLM outputs cannot be compared with a single scalar metric. Production evaluation uses a combination of: (1) **LLM-as-judge** — a capable model (e.g., Claude Opus) scores generated outputs against criteria; (2) **Reference-based** — BLEU/ROUGE for summarisation when gold references exist; (3) **Human annotation** — periodic sampling of prod outputs scored by humans and used to calibrate the LLM judge; (4) **Regression suites** — curated evaluation sets run on every prompt change.

Cost Tracking and Optimisation

PYTHON

```python
import anthropic

client = anthropic.Anthropic()

# Track token usage per request
response = client.messages.create(
    model="claude-sonnet-4-6",
    max_tokens=1024,
    messages=[{"role": "user", "content": prompt}]
)

input_tokens  = response.usage.input_tokens
output_tokens = response.usage.output_tokens

# Pricing as of 2026 (always verify at anthropic.com/pricing)
PRICE_PER_MTok_INPUT  = 3.00   # USD per million tokens
PRICE_PER_MTok_OUTPUT = 15.00

cost_usd = (input_tokens * PRICE_PER_MTok_INPUT +
            output_tokens * PRICE_PER_MTok_OUTPUT) / 1_000_000

log_to_cost_dashboard(user_id, cost_usd, input_tokens, output_tokens)
```

✦ **LLMOps cost levers**

(1) *Prompt caching* — Anthropic's prompt caching reduces cost for repeated system prompts by up to 90 %. (2) *Model routing* — use cheaper/faster models (Haiku) for simple classification; escalate to larger models only when needed. (3) *Output length control* — enforce structured outputs (JSON) to reduce verbose preambles. (4) *Batch API* — for non-latency-sensitive workloads, batch requests at 50 % discount. (5) *Caching at the application layer* — semantic cache identical or near-duplicate queries.

LLMOps Tooling Quick Reference

Q&A: Test Your Understanding

Q1. What is training-serving skew and how does it cause silent production failures?

Training-serving skew occurs when the feature computation used during model training differs from that used at serving time. For example, training computes "average spend last 30 days" on historical data using Spark, but serving computes the same feature differently in a microservice. The model was never trained on what it actually receives in production, so accuracy degrades invisibly without any error being thrown. The fix is a feature store: a single, shared computation path used by both training and serving.

Q2. Why is a scheduled retraining cadence (e.g., retrain every Monday) insufficient?

A fixed schedule retrains whether or not drift has occurred (wasting compute) and does *not* retrain fast enough when a sudden concept drift event hits mid-week (e.g., a market shock, viral trend, or fraud tactic change). Event-driven retraining — triggered by monitored drift thresholds — is more efficient and more responsive. In practice, most mature systems combine both: a scheduled safety-net weekly retrain plus a drift-triggered emergency retrain.

Q3. Explain the difference between data drift and concept drift with a fraud detection example.

Data drift (covariate shift): the distribution of input features changes. In fraud detection, a new payment method becomes popular and the "payment_type" feature distribution shifts. The model was never trained on this new category. **Concept drift**: the relationship between features and the target changes. Fraudsters adapt their tactics, so transactions with the *same* features that were benign last quarter are now fraudulent. Data drift can be detected without labels; concept drift requires labelled ground truth (or a proxy metric) to detect.

Q4. What is the purpose of a model signature in MLflow and when is it enforced?

A model signature defines the expected input schema (column names, dtypes, shapes) and output schema. It is enforced at *serving time* when a request arrives: if the payload does not match the signature, MLflow's serving layer raises a 400 error instead of passing malformed data to the model, which would produce a silent wrong prediction. This catches training-serving skew at the schema level early. Signatures are inferred from sample input/output at log time: `mlflow.models.infer_signature(X_train, model.predict(X_train))`.

Q5. What is shadow deployment and what specific questions does it answer before a canary release?

In shadow deployment, the new model receives a *copy* of every live production request and produces predictions, but those predictions are discarded — users see only the existing model's output. It answers: (1) Does the new model crash on any real-world input it wasn't tested on? (2) Are its prediction latencies acceptable under real production load? (3) Does its prediction distribution diverge from the production model in unexpected ways? Only after shadow validation passes does the team proceed to a canary (routing a small percentage of *live* traffic to the new model).

Q6. Why should model weights not be baked into a Docker image, and what is the recommended alternative?

Embedding model weights makes images multi-GB, causing multi-minute CI/CD push/pull cycles, bloated container registries, and requiring a full image rebuild every time weights are updated. The recommended pattern is a lean base image containing only the serving code and its Python dependencies, with weights loaded at startup from a model registry (S3, GCS, MLflow artefact store) using an environment variable pointing to the artefact URI. The image is rebuilt on code changes; the model is rolled forward by updating the environment variable in the Kubernetes Deployment.

Q7. What is the Population Stability Index (PSI) and what thresholds indicate action?

PSI measures how much a feature's distribution has shifted between a reference (training) dataset and a current (production) dataset. It bins the reference distribution and computes: PSI = Σ (% reference − % current) × ln(% reference / % current). Thresholds: PSI < 0.1 = no significant change; 0.1–0.2 = slight shift, monitor closely; > 0.2 = significant shift, trigger alert and consider retraining. PSI works on continuous features (after binning) and directly on categorical ones. It is symmetric and interpretable but sensitive to bin choice.

Q8. How does DVC's point-in-time data retrieval prevent label leakage during model training?

Label leakage occurs when features used at training time were computed using information available only after the event being predicted (e.g., using "account_closed_within_30_days" to predict churn, when that feature is unknown at prediction time). DVC itself stores data snapshots but does not compute features. The prevention of leakage requires a feature store's *point-in-time correct join* capability — the store retrieves feature values as they existed at a given entity timestamp, not their latest values. DVC ensures the exact dataset version used for a training run can be reproduced; the feature store ensures that dataset was assembled correctly.

Q9. What is "faithfulness" in the context of RAG evaluation and why is it not the same as accuracy?

Faithfulness (or groundedness) measures whether the LLM's generated answer is fully supported by the retrieved context — i.e., does the model invent facts not present in the passages? A model can be faithful (everything it says is in the context) but still inaccurate if the retrieved context itself contains wrong information. Conversely, a model can produce an accurate answer by hallucinating from training knowledge, which scores low on faithfulness. For RAG systems, faithfulness is often more important than accuracy because it measures whether the system is reliably using the provided source of truth rather than relying on potentially stale or incorrect parametric knowledge.

Q10. What are the three conditions that must be met for an ML experiment to be considered reproducible?

(1) **Same code**: the exact Git commit SHA must be recorded; any uncommitted change breaks reproducibility. (2) **Same data**: the DVC commit hash must point to the exact dataset, including preprocessing scripts; the data must not have been re-fetched or re-processed with updated logic. (3) **Same environment**: the

> Docker image digest (or conda lock file hash) must be pinned; different library versions — even patch versions — can change numerical results. Additionally, all random seeds must be fixed and any non-deterministic GPU operations disabled. MLflow's run metadata should store all four identifiers (Git SHA, DVC hash, image digest, seed) at training time.

MLOps Tooling Landscape

The MLOps ecosystem is broad and fast-moving. This table maps the major concerns to mature, widely-used tools as of 2026.

Concern	Open-Source Tools	Managed / Cloud	Notes
Experiment tracking	MLflow, Weights & Biases (OSS)	W&B Cloud, Comet ML, Neptune.ai	MLflow is most portable; W&B has best UI for deep learning
Data versioning	DVC, LakeFS	Databricks Unity Catalog	DVC for files/datasets; LakeFS for Delta/Iceberg lake branching
ML pipelines	Kubeflow Pipelines, Prefect, ZenML, Metaflow	Vertex AI Pipelines, SageMaker Pipelines, Azure ML Pipelines	Choose based on your orchestration maturity and cloud affinity
Model registry	MLflow Registry, BentoML Models	Vertex AI Model Registry, SageMaker Model Registry, HF Hub	Critical to have before going to production
Feature store	Feast, Hopsworks (community)	Tecton, Vertex AI Feature Store, SageMaker Feature Store	High upfront cost; highest ROI in companies with 5+ ML models
Model serving	BentoML, TorchServe, TF Serving, Triton, Ray Serve	Vertex AI Endpoints, SageMaker Endpoints, Azure ML	Triton for GPU; BentoML for ease of use; Ray Serve for Python-native scaling
Data validation	Great Expectations, Pandera, TFDV	Monte Carlo, Bigeye	Run in CI and as a pipeline pre-check before every training run
Drift monitoring	Evidently AI, NannyML, Alibi Detect	Arize AI, Fiddler AI, WhyLabs	NannyML excels at monitoring without ground truth labels
LLM tracing & monitoring	Langfuse (OSS), Arize Phoenix, OpenTelemetry	LangSmith, Helicone, PromptLayer	Full span tracing essential for multi-step LLM chains
LLM evaluation	RAGAS, DeepEval, TruLens, Promptfoo	W&B Weave, Braintrust	RAGAS for RAG; DeepEval for general LLM eval with custom metrics
Containerisation	Docker, Kaniko (CI), BuildKit	ECR, GCR, ACR, GHCR	Kaniko for rootless builds in Kubernetes CI runners
Kubernetes ML platform	Kubeflow, MLRun	Vertex AI, SageMaker, Azure ML, Databricks	Managed platforms trade flexibility for ops simplicity
CI/CD for ML	GitHub Actions, GitLab CI, DVC + CML	GitHub Actions + cloud runners	CML (Continuous Machine Learning) by Iterai adds ML-specific GitHub Actions
A/B / experimentation	Unleash, Flipt	LaunchDarkly, Split.io, Optimizely	Use a feature flag system; never implement your own traffic split logic

✦ **Stack selection principles**
(1) **Start minimal**: Git + DVC + MLflow + Docker + a monitoring tool covers 80 % of teams' needs. (2) **Avoid vendor lock-in** in the critical path: the model registry, serving layer, and pipeline orchestrator should be portable. (3) **Buy the undifferentiated heavy lifting**: GPU infrastructure, logging, and alerting are commodity; feature stores and evaluation frameworks are where teams build competitive advantages. (4) **Add a feature store only after** you have two or more teams training models on overlapping data.